BFOR516 - Recurrent Neural Networks

Srishti Gupta, Ph.D.

Recurrent Neural Networks (RNNs)

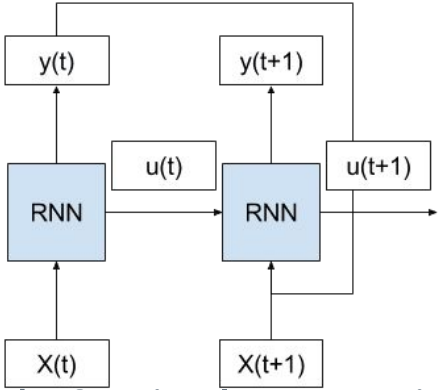
- RNNs are a type of deep learning model designed to handle sequential data. Unlike traditional neural networks, which process all inputs independently, RNNs have loops (or recurrent connections) that allow information to be passed from one step to the next.
- Imagine you are reading a sentence word by word. Instead of forgetting the previous words, your brain keeps track of context. RNNs work the same way—they process one element at a time while maintaining a memory of previous elements.

Why Are RNNs Important?

- RNNs became widely used in the 2010s for tasks like:
  - Handwriting recognition (Graves et al., 2008)
  - Machine translation (Sutskever et al., 2014)
  - Medical diagnosis prediction (Lipton et al., 2016)
- However, RNNs have some limitations, such as struggling with long sequences due to memory loss over time (the "vanishing gradient problem"). More advanced models, like LSTMs (Long Short-Term Memory networks) and Transformers, were later developed to handle these issues.

Unrolling an RNN

- Because of their looped structure, RNNs can be "unrolled" to visualize them as a sequence of repeated neural network layers, where each step takes the previous step's output as an input. This helps the model retain past information while making predictions.



Step 1 Step 2 Step 3 Step 4

[I] → [love] → [deep] → [learning!]

⬆️ ⬆️ ⬆️

(Memory passed forward)

- Instead of a single repeating loop, we can visualize the RNN as multiple copies of the same neural network applied to different time steps.

Understanding Sequential Data

- Instead of a single input, we now have a sequence of inputs, such as:

$$X = [X_1, X_2, X_3, \ldots\ldots X_T]$$

- Where:
  - X_T is the data at time step t (e.g., a word in a sentence, a stock price on a given day).
  - T is the length of the sequence.
- Some datasets are one long sequence (e.g., a stream of temperature readings). Others consist of many sequences (e.g., multiple patient records, each with its own length).
- Why is this important?
  - The order of data points matters.
  - The future often depends on the past (e.g., a movie review's sentiment is influenced by earlier words).

Types of Sequence Learning Tasks

Depending on what we want to predict, we classify sequence tasks into different types:

1. **Sequence-to-label (Fixed target)**
   a. Example: Sentiment classification of a review (positive or negative).
   b. Input: A sequence (words in a review)
   c. Output: A single label (positive/negative)
2. **Input-to-sequence (Fixed input, sequential output)**
   a. Example: Image captioning
   b. Input: A single image
   c. Output: A sequence (words forming a caption)
3. **Sequence-to-sequence (Sequential input, sequential output)**
   a. Example: Machine translation (English to French)
   b. Input: A sequence (English sentence)
   c. Output: A sequence (French sentence)
   d. **Aligned sequence-to-sequence:** Each input corresponds to a specific output (e.g., part-of-speech tagging).
   e. **Unaligned sequence-to-sequence:** No strict one-to-one correspondence (e.g., translating "How are you?" to "Comment ça va?" in French, where word alignment isn't exact).

Sequential Modeling - Autoregressive Models

Key idea: Use past values to predict the future from the same series.

Example: Stock price prediction

- Imagine you want to predict tomorrow's stock price. You have past prices:
  - (Day 1: $100, Day 2: $102, Day 3: $101, ... Day t: $105)
  - An autoregressive model will look at these past values and predict Day t+1's price.
- Challenges:
  - How many past values should we use?
  - What if there's a lot of noise?
  - Longer history means more computations!
- Two Common Strategies:
  - Fixed Window Size – Use only the last k time steps instead of the full history.
    - Example: Use only the past 7 days of stock prices to predict tomorrow's price.
  - Latent State Representation – Maintain a hidden state (summary) instead of storing the full history.

Sequential Modeling - Language Modeling

A language model is a mathematical tool used to predict the likelihood of a sequence of words in a language. In simple terms, it helps determine how "natural" or "probable" a given sentence is.

The **goal of a language model** is to estimate the probability of a sequence of words occurring together. Mathematically, if we have a sentence of words:

$$P(w_1, w_2, \ldots, w_T)$$

we want to predict the probability of the entire sequence being valid.

To simplify, we use **conditional probabilities**:

$$P(w_1)P(w_2 \mid w_1)P(w_3 \mid w_1, w_2)\ldots$$

Each word is predicted based on previous words.

Sequential Modeling - Language Modeling

Example: Predicting the next word in a sentence

- Imagine you're texting and your phone suggests words:
  - "The cat sat on the ___"
  - Your phone predicts "mat" instead of "banana" because it has learned common patterns in language.
- How does it work?
  - We model the probability of a sequence:
  - P("mat" | "The cat sat on the")
  - The model learns that some words follow others more often than random choices.
- **Markov Models (Simplified Memory)**
  - One way to model this is to only remember the last few words instead of the entire history. Example: Predict X_t using only $X_{(t-1)}$ and $X_{(t-2)}$ (instead of the whole sentence). This is called the Markov assumption—the idea that the present only depends on a fixed number of past steps.

Sequential Modeling - Language Modeling

- **Word Frequency & Probability (n-gram)**
  - To compute probabilities, we count word occurrences in the dataset.
  - For a single word (Unigram Model):

$$P(w) = \text{count of } w/\text{total word count}$$

  - For two words together (Bigram Model):

$$P(w_2|w_1) = \text{count of } (w_1,w_2)/\text{count of } w_1$$

  - But rare word pairs create problems (some combinations may never appear in training data).

Sequential Modeling - Language Modeling

- **Laplace Smoothing**
  - To avoid assigning zero probability to unseen words, we use Laplace Smoothing:

$$P(w) = \text{count of } w + \alpha \,/\, \text{total words} + \alpha V$$

  - where:
    - α is a small number (e.g., 1),
    - V is the total number of unique words.
  - This ensures that every word has a small, nonzero probability.

Sequential Modeling - Language Modeling

- **Perplexity (Measuring Model Performance)**
  - How do we know if our language model is good? We use perplexity, a measure of how well the model predicts text.
    - A lower perplexity means the model is better at predicting words.
    - A higher perplexity means the model is uncertain.
  - If a model predicts "It is raining outside" with high probability and "It is raining banana tree" with low probability, it has low perplexity (good performance).
  - Perplexity formula:

$$\textbf{Perplexity} = 2^{\text{cross-entropy loss}}$$

  - This helps compare models on different text lengths.

Sequential Modeling - Language Modeling

Traditional models like n-grams have limitations:

- They struggle with long sentences.
- They require storing large word counts.
- Deep learning (e.g., RNNs, Transformers) improves language models by learning word relationships in context, making predictions smarter and more accurate.

Converting Raw Text into Sequence Data

1. Loading the Text
   a. The text is first read into the computer as a string (a sequence of characters).

```
text = "The Time Machine, by H. G. Wells [1898]"
print(text)
```

```
The Time Machine, by H. G. Wells [1898]
```

2. Preprocessing the Text
   a. All punctuation is removed.
   b. All letters are converted to lowercase to keep things consistent.

```
import re

def preprocess(text):
    return re.sub('[^A-Za-z]+', ' ', text).lower()

clean_text = preprocess(text)
print(clean_text)
```

```
the time machine by h g wells
```

Converting Raw Text into Sequence Data

3. Tokenization (Splitting Text into Small Units)
   a. The text is split into smaller units called tokens. These tokens can be:
      i. Words (e.g., "time", "machine", "traveller")
      ii. Characters (e.g., 't', 'i', 'm', 'e', ' ')

```
char_tokens = list(clean_text)
print(char_tokens[:10])  # Print first 10 characters
```

```
['t', 'h', 'e', ' ', 't', 'i', 'm', 'e', ' ', 'm']
```

```
word_tokens = clean_text.split()
print(word_tokens)
```

```
['the', 'time', 'machine', 'by', 'h', 'g', 'wells']
```

Converting Raw Text into Sequence Data

4. Building a Vocabulary
   a. The unique tokens (characters or words) in the text are collected.
   b. Each unique token is assigned a number (an index).
   c. If a token is not in the vocabulary (rare or unseen words), it is replaced with a special <unk> token (unknown token).

```python
from collections import Counter

# Count occurrences of each word
word_counts = Counter(word_tokens)
vocab = {word: idx for idx, (word, _) in enumerate(word_counts.items())}

print(vocab)
```

```
{'the': 0, 'time': 1, 'machine': 2, 'by': 3, 'h': 4, 'g': 5, 'wells': 6}
```

Converting Raw Text into Sequence Data

5. Converting Text into Numbers
   a. The entire text is transformed into a list of numbers where each number corresponds to a token in the vocabulary.

```
numerical_sequence = [vocab[word] for word in word_tokens]
print(numerical_sequence)
```

```
[0, 1, 2, 3, 4, 5, 6]
```

6. Exploring Word Frequency
   a. The most common words in the dataset are counted and listed.
   b. Common words like "the", "and", and "to" appear frequently but are not very meaningful for understanding the content. These are called stop words.

```
print(word_counts.most_common(3))  # Top 3 words
```

```
[('the', 1), ('time', 1), ('machine', 1)]
```

Recurrent Neural Networks (RNNs)

1. **Markov Models and n-grams for Language Modeling**
   a. In language modeling, we try to predict the next word in a sequence of words based on the previous ones.
   b. Markov Models and n-grams are simple methods where the prediction only depends on the previous word or a small number of previous words (n-grams). For example, a bigram model predicts the next word only based on the previous word.
   c. However, as you want to consider more than just the immediate previous tokens, you would need to increase the number of past tokens (i.e., use longer sequences). But doing so exponentially increases the number of parameters you need to store, which is impractical for large vocabularies.

Recurrent Neural Networks (RNNs)

2. **Hidden States and Latent Variables**
   a. In the context of RNNs, the hidden state represents the memory of the network, storing information about the sequence of tokens observed so far.
   b. RNNs use hidden states to model sequential data. At each time step, the network not only looks at the current input but also incorporates the hidden state from the previous time step.
   c. The hidden state stores the relevant information about the sequence that has been processed so far. The hidden state at each time step is updated based on both the input at that time step and the hidden state from the previous step.
   d. To avoid the exponential growth of model parameters, **latent variables** can also be used. These variables are not directly observed but are inferred from the observed data.

Recurrent Neural Networks (RNNs)

3. **ANN vs RNN**
   a. In a traditional neural network like a multilayer perceptron, each layer is independent of previous layers. The output of each layer depends only on the current input, and the model doesn't have a memory of previous inputs.
   b. In an RNN, the hidden state (memory) allows the model to capture and retain information from previous time steps, making it more suited for sequential data.
4. **RNNs with Hidden States**
   a. For RNNs, at time step t, the hidden state h_t is computed using:

$$h_t = f(x_t, h_{t-1})$$

   b. Where: x_t is the input at time step t, h_{t-1} is the hidden state from the previous time step, f is a function that updates the hidden state.
   c. The hidden state at each time step depends on both the input at that step and the hidden state from the previous step. This allows RNNs to model sequences where the current state is influenced by past events.

Recurrent Neural Networks (RNNs)

5. **Computational Example**

- Let's say we have a sequence of inputs at each time step. At each time step, the input and the previous hidden state are processed to generate the new hidden state.
- This is often expressed mathematically as: $h_t = f(W_{xh}x_t + W_{hh}h_{t-1} + b)$
  - h_t: This is the new **hidden state** at time step t. It's like the memory of the network at this step, which holds all the information the network has learned from the previous inputs.
  - x_t: This is the **current input** at time step t. For example, if you're working with text, this could be the word the network is currently analyzing.
  - h_{t-1}: This is the **previous hidden state** from the previous time step t−1. It represents the memory from before, which helps the network remember what it learned earlier.
  - W_{xh}: This is the **weight matrix** that connects the current input x_t to the hidden state. It decides how much influence the current input has on the new hidden state. Think of it as a "weight" or "importance" factor that controls how much the current input matters.
  - W_{hh} This is another **weight matrix** that connects the previous hidden state h_{t-1} to the new hidden state hth_tht. It controls how much the previous memory should influence the current memory. Essentially, it lets the network "remember" past information.
  - **b**: This is the **bias** term. It's a constant added to the equation to help the model learn better. Without the bias, the model might struggle to fit the data. It just shifts the result a little bit, giving the network more flexibility.
  - **f**: This is the **activation function**. It's usually a non-linear function (like **tanh** or **ReLU**) that helps the network learn complex patterns. It ensures that the output (the new hidden state hth_tht) is within a certain range or has a specific shape. Without this function, the model would only learn linear patterns, which is limiting.

Recurrent Neural Networks (RNNs)

6. **Output of the RNN:**
   a. After calculating the hidden state for each time step, we use it to produce an output. For instance, in the context of language modeling, this output might be the prediction of the next word.
   b. The output at each time step is computed similarly to a traditional neural network:

$$y_t = g(W_{hy} h_t + c)$$

   c. Where:
      i. W_{hy} is the weight matrix from the hidden state to the output.
      ii. c is the bias.
      iii. g is the output activation function

Backpropagation Through Time (BPTT)

- **RNN Structure**: In an RNN, each time step has a hidden state that is updated based on the input at that time and the hidden state from the previous time step. The hidden state carries information forward from one step to the next.
- **Unrolling**: To apply backpropagation to RNNs, we "unroll" the network across all time steps. This means we treat each time step like a layer in a feedforward neural network, with shared parameters across all time steps. This helps in calculating gradients for each parameter by applying the chain rule through each time step.

Backpropagation Through Time (BPTT)

- **Gradient Computation:**
  - Forward Propagation: Just like in other neural networks, the first step is forward propagation where the model predicts outputs based on inputs at each time step.
  - Backward Propagation (BPTT): Once the forward pass is done, we compute the error (or loss) by comparing the predicted outputs with the actual targets. BPTT then computes the gradients for all the model parameters by applying the chain rule, starting from the final time step and moving backwards through time.

Backpropagation Through Time (BPTT)

- **Vanishing and Exploding Gradients:**
  - When sequences are long, gradients can either vanish (become too small) or explode (grow too large), making training difficult. This happens because, as we go further back in time, the gradients can either shrink or grow exponentially depending on the weights and activation functions.
  - Vanishing gradients: Small gradients make it hard for the model to learn from earlier time steps.
  - Exploding gradients: Very large gradients can destabilize the model, causing it to fail during training.

Backpropagation Through Time (BPTT)

- **Strategies to Address Gradients Issues:**
  - **Truncated BPTT:** Instead of considering all time steps, we limit the backpropagation to a certain number of steps. This prevents the gradients from becoming too large or too small, making training more stable. This is often used in practice, as it balances between computation time and performance.
  - **Randomized Truncation**: This technique involves truncating the sequence randomly, with some time steps being kept longer than others. While it's more complex, it can sometimes lead to better performance by focusing on different time steps in a randomized manner.

Backpropagation Through Time (BPTT)

- **Backpropagation Derivation:**
  - The key challenge in BPTT is calculating how the error in the final output affects the weights throughout all the previous time steps. This requires applying the chain rule repeatedly for each time step, which is what leads to the gradient computation.
  - Gradients are calculated by recursively applying the chain rule for each parameter, starting from the final time step and moving backward through time. This gives us the gradient at each step and allows us to update the model parameters.

Backpropagation Through Time (BPTT)

- **Example:**
  - Imagine an RNN used for sentiment analysis over a sentence:
    - The RNN reads the words in the sentence one by one and updates its hidden state with each new word.
    - At the end, the RNN produces an output representing the sentiment (positive/negative/neutral).
    - Backpropagation computes how much each word (and its hidden state) contributed to the final sentiment. The gradients are calculated for each weight across all time steps, updating the model to better predict the sentiment.